

COLORADO STATE UNIVERSITY PUEBLO

School of Engineering

A Graph Neural Surrogate for Longitudinal Train Dynamics

Committee Progress Update

George James

September 2026



Where Things Stand

Thesis chapters

- | | |
|-------|---|
| 1–6 | Written; revised this month to the current design |
| 7 | GPU integrator and dataset |
| 8 | Surrogate architecture |
| 9 | First draft: training |
| 10 | First draft: evaluation |
| 11–12 | Not started: control, conclusion |
-

Built and working

- Reference simulator, route pipeline and RailLab console
- GPU simulator and two 10,000-scenario datasets
- The surrogate, trained and tested over 200-step rollouts

111 pages

Compiled thesis, chapters 1–8

10,000

Scenarios per dataset, 10 to 150
vehicles

Not built

Control demonstration

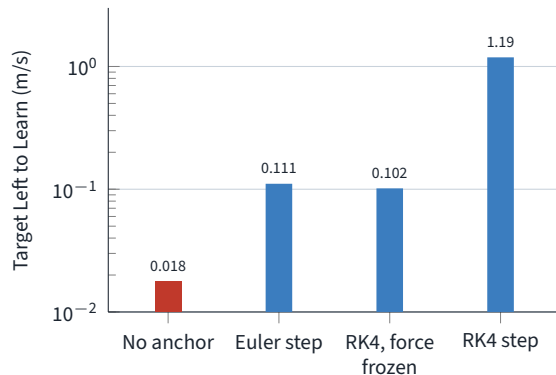
The Surrogate Design Changed

The original plan combined three ideas. One was kept and two were dropped.

Part of the original plan	Status	Reason
Graph neural network: vehicles as nodes, couplers as edges	Kept	Handles any train length with one set of weights.
Learn a correction to the physics equations	Dropped	The training data come from those same equations, so the correction is exactly zero.
Neural ODE: learn a derivative and integrate it with a solver	Dropped	Stiff couplers would force 5 or more network calls per stored step, between states no data can check.

New design: a graph network simulator. The network predicts one fixed 0.25 s step. One network call replaces 50 evaluations of the physics equations, and more of the physics is built in than before, not less.

Why: The Couplers Are Stiff at This Step Size



Median over 480 transitions of data/v2. Lower is an easier learning problem.

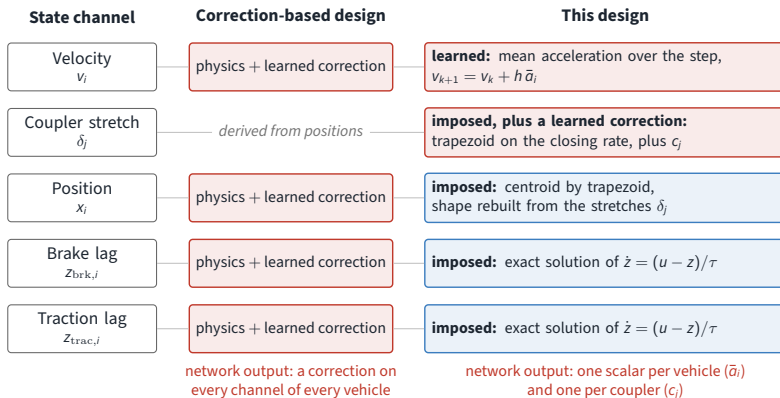
An **anchor** is a cheap physics estimate of the next state; the network would learn only what is left over.

For velocity, **every physics anchor leaves more to learn**, about $6\times$ more at best. One RK4 step of 0.25 s diverges.

The cause is stiffness: a coupler between two heavy cars oscillates with a period as short as 0.21 s, and an explicit step of 0.25 s is unstable for it ($\omega h = 7.45$ against a limit of about 2.8).

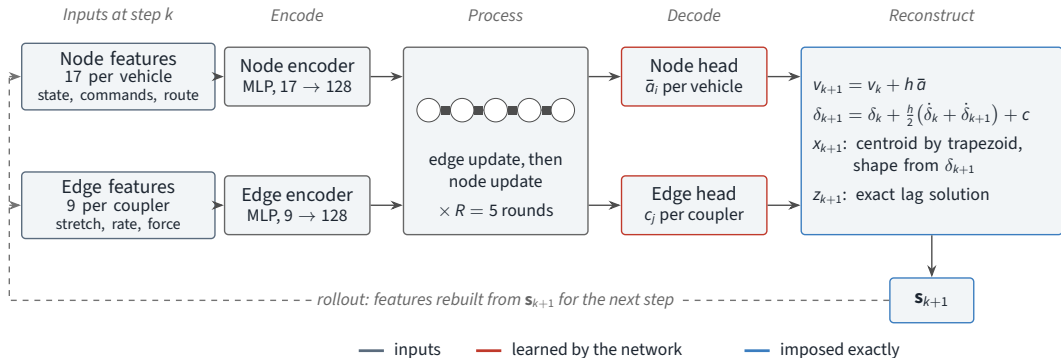
For position the anchor helps: $\dot{x} = v$ exactly, so it is **113 $\times$ better**.

What the Network Predicts, and What Is Computed Exactly



The brake and traction lags have an exact solution, and position follows from velocity. The network predicts **two numbers**: a mean acceleration per vehicle and a shape correction per coupler.

The Network



730,370

parameters, the same for any
train length

5 rounds

set by how far a force wave
travels in 0.25 s

1 call

per 0.25 s step, replacing 50

Data: The GPU Simulator and the Route Fix

GPU simulator

~**170 CPU-h** → **25 min**

Time to build a 10,000-scenario dataset

Flat in N

A 150-car train costs the same as an 11-car one

Agrees with the original simulator within 0.05 m/s and 5% on peak coupler force.

Route fix

- One of the five corridors had elevation samples that were not on the track: 19.3% of its 10 m steps jumped more than a metre.
- It is now rejected by a check on the raw elevation. Four corridors remain.
- The larger corridor search failed on OpenStreetMap and was switched to the national rail network dataset: 274,145 km of main line.
- **Both datasets were built before this fix**, so they are being rebuilt as we speak.

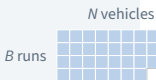
How the GPU Simulator Works

Same equations as the original simulator. Four changes make it fast enough to build the datasets.

1. Whole trains at once

The original loops over vehicles one at a time in Python. The GPU version holds every vehicle of every run in one table (a *tensor*) and computes each force for all of them at once. Shorter trains are padded with empty vehicles (white).

Cost flat in N



2. A fixed time step

0.02 s step

The original solver shrinks its step whenever any coupler reaches the edge of its slack: on a long train, almost always. The GPU version uses a standard fixed-step method (RK4), so every run takes a known number of steps and none can stall. States are saved every 0.25 s.

3. Record one step, replay it

10× faster

Each step was about 660 small GPU jobs, and the GPU sat idle 92 % of the time waiting for the next one. The step is now recorded once (a *CUDA graph*) and replayed as one job: 38 s down to 3.8 s.

4. Extra digits where needed

1.2× the cost

Calculations keep about 16 digits (*double precision*); results are saved with about 7 (*single*). With 7 digits, rounding alone used 60 % of the allowed speed error. 16 digits cost only 20 % more time.

Checked against the original with 60 tests on 15 reference scenarios, written before the GPU version: speed within 0.05 m/s, couplers engage and release at the same moments in ≥ 98 % of samples, peak coupler force within 5 %.

How the Results Are Measured

The test

- Trained on trains of 10 to 80 cars. Tested on trains of 121 to 150 cars that it never saw.
- **Rollout:** the model predicts 0.25 s ahead, then starts the next step from its own prediction. 200 steps covers 50 s.
- **Error:** the worst vehicle (or coupler) in the train, compared with the simulator's true trajectory. The typical (median) value over many random starting points is reported.

The reference: “hold”

- Keep the starting state and never update it.
- Its error is therefore **how much the train actually changed**. It measures how much there was to predict; it is a scale, not a competing model.
- Results are stated as **model error as a share of the actual change**: 0% is perfect, 100% is no better than standing still.

Real comparison models (constant-acceleration extrapolation, a physics-correction MLP, a transformer) are planned for the evaluation chapter and have not been run yet.

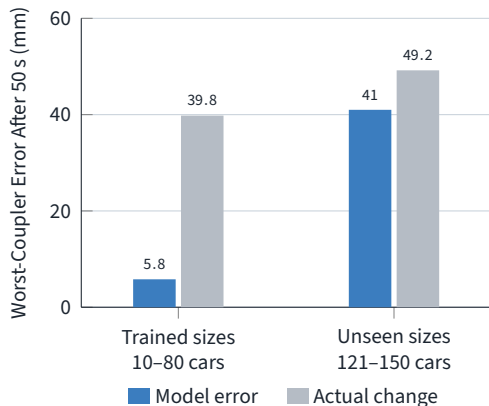
Result: Speed Stays Accurate on Longer Trains

Rollout	Trained sizes, 10–80 cars			Unseen sizes, 121–150 cars		
	Actual change	Model error	Share	Actual change	Model error	Share
1 step, 0.25 s	0.016	0.004	23%	0.028	0.011	39%
50 steps, 12.5 s	0.48	0.028	5.7%	0.53	0.061	11%
200 steps, 50 s	1.78	0.066	3.7%	1.31	0.118	9.0%

Speed of the worst vehicle, in m/s. “Actual change” is the error of holding the starting state.

- After 50 s, the model’s speed error is **0.066 m/s** on familiar train lengths and **0.118 m/s** on trains up to 15 times longer: under a tenth of how much the speed actually changed.
- The one-step share is higher only because a train barely changes in a quarter second. The long rollouts are what a simulation uses.
- Doubling training time nearly halved the 200-step error, so the model is not yet fully trained.

Result: Coupler Shape Does Not Hold Up on Longer Trains



Coupler shape is how far each coupler is stretched or compressed. Past the 20 mm of slack, every extra millimetre adds force.

On familiar lengths the model's error is **15%** of the actual change. On the long trains it is **83%**: almost as large as the change itself.

The control chapter limits **coupler force**, on long trains, over long stretches of time. This is the result that most affects the remaining work.

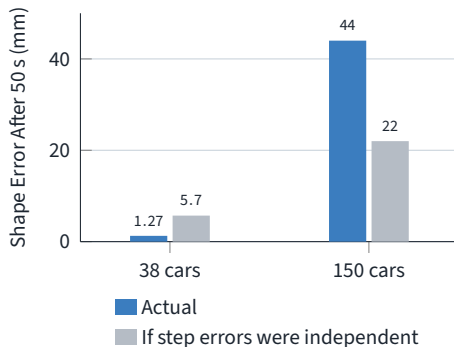
Why Coupler Shape Fails: Errors Build Up Instead of Cancelling

Three explanations tested and ruled out

- **Too few message-passing rounds?** No: error is even along the train, not larger far from the ends.
- **A slow stretch of the whole train?** No: removing it made familiar lengths 47% worse and long trains no better.
- **Caused by the speed error?** No: with perfectly correct speeds, only 9% of the shape error went away.

Per step, the model is nearly as accurate on long trains (17% worse). The problem appears only over many steps.

Right: one diagnostic run on a 38-car and a 150-car train.



On a familiar length the errors **cancel**; on a long train they **build on each other**. A stability problem, not an accuracy problem.

- 1 No corridors held out for the control demonstration.** With only four corridors, reserving one more leaves too little to train on.
- 2 The steep-grade test set is built on the rejected corridor,** so it tests false grades rather than real mountain grades.
- 3 The national rail network is not yet split into routes.** It is a graph of 95,936 segments; turning it into corridors is the next piece of code. Everything above waits on it.
- 4 Coupler shape on long trains** (slides 10 and 11), which matters most for the control chapter.

Planned fix for 1 and 2: extract corridors including real steep crossings such as Raton Pass, then rebuild with 50,000 to 100,000 scenarios, about 2 to 4 hours on the GPU. Results so far will be re-run on the new data.

Questions for the Committee

- 1 Overspeed runs.** About 18% of scenarios exceed the speed limit by more than 5 m/s, because the simulator does not enforce it. Remove them from training, or keep them as hard cases?
- 2 Control demonstration scope.** Train the control model on all train lengths, which avoids the shape problem, and hold out corridors instead of train lengths?

Next steps, in order

- 1 Split the national rail network into corridors
- 2 Rebuild the dataset on the corrected, larger corridor set
- 3 Retrain and re-run the results
- 4 Build the control demonstration

Remaining writing

- 9 Training: first draft done; update for the rebuilt data
- 10 Evaluation: first draft done; re-run results on the rebuilt data
- 11 Control demonstration: not started
- 12 Discussion and conclusion: not started